HyperCluster: Decentralized Large Language Model Inference over Peer-to-Peer Wireless Networks

Anonymous Submission

Affiliations Placeholder

Abstract. The substantial memory and computational requirements of large language models (LLMs) preclude their deployment on individual resource-constrained wireless devices. This paper introduces HyperCluster, a framework for fully decentralized collaborative inference of LLMs over peer-to-peer wireless networks. Our system contributes four core innovations: (1) a ring-based pipelined inference protocol where nodes deterministically self-organize into a computational ring based on device capabilities, passing intermediate activations directly between peers using QUIC-based content-addressed networking; (2) a generalizable model sharding engine built upon the Hugging Face Transformers library that automatically partitions any dense LLM across heterogeneous devices proportional to available memory; (3) a direct QUIC transport layer inspired by the Lattica framework, employing persistent connections, binary-framed RPC, and connection pre-warming to minimize inter-node transfer latency; and (4) a tail-node sampling optimization that reduces the return-path data transfer by approximately 18,000× per generation step. We provide a practical validation on a heterogeneous cluster of three consumer-grade devices communicating over separate networks via the Iroh P2P stack, demonstrating correct distributed inference of models with up to one billion parameters and identifying the key performance bottlenecks that inform future optimization efforts.

Keywords: Distributed AI · Peer-to-Peer Networks · LLM Inference · Model Sharding · Ring Pipeline · QUIC Transport

1 Introduction

The proliferation of large language models (LLMs) has created an unprecedented demand for computational resources. State-of-the-art models require tens to hundreds of gigabytes of memory, far exceeding the capacity of typical consumer and edge devices. While cloud-based services provide convenient access, they introduce latency, privacy concerns, and dependence on centralized infrastructure—issues that are particularly acute in wireless and edge computing scenarios where connectivity may be intermittent or bandwidth-constrained.

An alternative paradigm is *collaborative inference*, where multiple devices in a peer-to-peer (P2P) network pool their resources to collectively execute a model

that no single device could run alone. However, realizing this vision in a decentralized wireless environment presents unique challenges. Data-center frameworks such as Megatron-LM [1] and DeepSpeed [2] presuppose high-bandwidth, low-latency interconnects (NVLink, InfiniBand) and stable cluster memberships—assumptions that fundamentally do not hold in dynamic P2P networks where nodes communicate over heterogeneous wireless links with variable latency and bandwidth.

Several recent systems have explored decentralized inference. Petals [4] enables collaborative inference through voluntary GPU contributions, but relies on centralized coordination for peer discovery. Exo [5] supports heterogeneous "home clusters" but is limited to local networks without NAT traversal. Prima.cpp [8] introduces an efficient ring pipeline but targets static cluster configurations. None of these systems simultaneously address *internet-scale NAT traversal*, *broad model compatibility* via standard libraries, and *communication optimization* for bandwidth-constrained links.

This paper introduces **HyperCluster**, a framework for fully decentralized collaborative inference designed for heterogeneous devices communicating over P2P networks across the internet. Built on the Iroh [9] content-addressed networking stack, HyperCluster enables nodes separated by NATs and firewalls to self-organize into a computational ring and collectively execute standard Hugging Face Transformer models without any model-specific modifications.

Contributions. This paper makes the following contributions:

1. **Decentralized Ring-Pipeline Protocol:** A coordinator-free inference protocol where nodes self-organize into a deterministic computational ring based on advertised device capabilities. Intermediate activations and synchronization metadata (position IDs, attention masks) flow directly between peers, enabling correct autoregressive generation without centralized coordination (§4).
2. **Generalizable Transformers-Based Sharding Engine:** A dynamic model sharding methodology that integrates directly with the Hugging Face Transformers library [3], enabling any dense, off-the-shelf LLM to be automatically partitioned across heterogeneous devices according to available memory. The engine employs runtime layer extraction with configuration patching to ensure correct KV cache allocation on each shard (§3.4).
3. **Direct QUIC Transport with Lattica-Inspired Framed RPC:** A high-performance tensor transfer layer using persistent QUIC connections, binary-framed msgpack RPC, and 4 MB chunked streaming. Connection prewarming during ring initialization eliminates first-transfer handshake latency of 6–20 ms (§3.2).
4. **Tail-Node Sampling Optimization:** An optimization where the tail node samples the next token locally and returns only the 8-byte token ID to the head node, rather than transmitting the full logit tensor ($\sim$0.58 MB for a 150K-vocabulary model), achieving an approximately 18,000$\times$ reduction in return-path data per autoregressive step (§4.3).

We validate HyperCluster on a heterogeneous cluster of three consumer-grade devices—a MacBook Air M2 (16 GB), a Mac Mini (8 GB), and an Intel i5 Linux laptop (8 GB)—communicating across separate networks via the Iroh P2P stack. Our experiments demonstrate correct distributed inference of Llama-3.2-1B-Instruct [10] and Qwen3-0.6B [11], and our analysis identifies the communication bottleneck as the dominant factor governing distributed inference throughput.

2 Background and Related Work

This section situates HyperCluster within three research areas: centralized model parallelism, decentralized inference systems, and communication topologies for distributed computation.

2.1 Model Parallelism in Centralized Environments

The challenge of partitioning large neural networks has been extensively studied within the high-trust, homogeneous environments of data centers. Megatron-LM [1] introduced tensor (intra-layer) and pipeline (inter-layer) parallelism for training multi-billion parameter models across GPU clusters. DeepSpeed [2] contributed ZeRO optimizer partitioning for memory-efficient training at scale. Alpa [12] automated the search for optimal inter- and intra-operator parallelism strategies. While these systems provide foundational techniques, they are architected for stable hardware with NVLink/InfiniBand interconnects—assumptions that are fundamentally incompatible with the bandwidth-constrained, heterogeneous P2P networks targeted by HyperCluster.

2.2 Decentralized and Peer-to-Peer Inference

Recent work has begun exploring inference workload distribution over less reliable, internet-scale networks:

Petals [4] demonstrated a collaborative system where users contribute GPU resources to a shared network for inference and fine-tuning. However, Petals relies on a central authority for peer discovery and routing, and assumes high-bandwidth, stable connections, limiting its applicability in dynamic wireless environments.

Exo [5] enables distributed inference across a "home cluster" of everyday devices, employing ring memory-weighted partitioning to split models based on device capabilities. While Exo's partitioning strategy influenced HyperCluster's approach, it is primarily designed for local networks, does not address NAT traversal for internet-scale collaboration, and supports only a limited set of model families.

Lattica [6] provides a universal, cross-NAT communication framework for decentralized AI, establishing a globally addressable P2P mesh using libp2p with noise+yamux transports. HyperCluster draws architectural inspiration from Lattica's framed RPC protocol design, adapting it to Iroh's native QUIC streams.

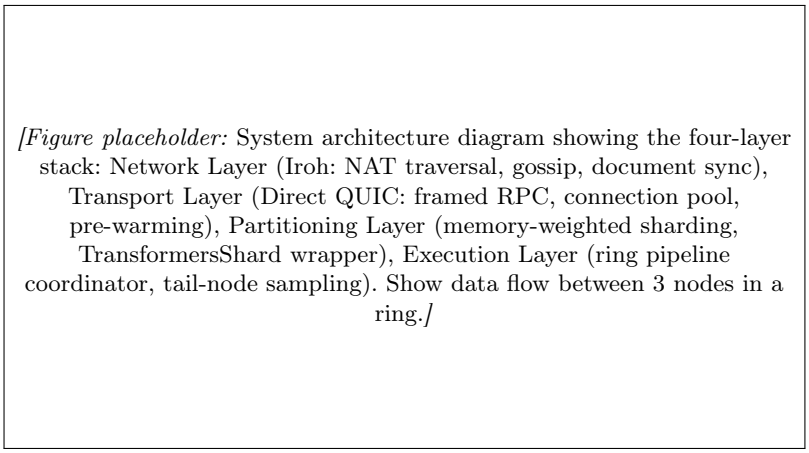

Fig. 1: HyperCluster system architecture. The four-layer stack enables decentralized ring-pipeline inference over P2P wireless networks.

Hyperspace explores a general-purpose P2P network for AI using Distributed Hash Tables (DHTs) and cryptographic protocols. HyperCluster builds upon these foundational P2P principles by integrating networking primitives directly into a purpose-built, application-level protocol for collaborative inference.

2.3 Ring Topologies for Distributed Computation

The ring all-reduce algorithm [7] is a foundational communication pattern for distributed training. Prima.cpp [8] adapted this topology for inference, demonstrating that a pipelined ring architecture can effectively split LLMs across heterogeneous devices by overlapping disk I/O with computation. HyperCluster extends the ring concept to a fully dynamic, geographically distributed wireless environment by implementing it on Iroh's [9] QUIC-based P2P stack, which provides built-in NAT traversal and connection management across firewalls.

3 System Architecture

HyperCluster is organized into four layers, depicted in Figure 1: a *Network Layer* for P2P communication and topology management, a *Transport Layer* for high-performance tensor transfer, a *Partitioning Layer* for dynamic model sharding, and an *Execution Layer* for ring-pipeline inference orchestration.

3.1 Network Layer: P2P Communication via Iroh

The network layer is built upon **Iroh** [9], a content-addressed networking library that provides three essential primitives for P2P environments:

- **NAT Traversal:** Iroh's QUIC-based endpoint establishes direct peer connections across firewalls using relay servers as a fallback, enabling nodes in separate networks to communicate without manual port forwarding.
- **Gossip Protocol:** Decentralized peer discovery and lightweight message broadcasting enable nodes to announce their presence and capabilities without centralized coordination.
- **Document Synchronization:** Iroh's CRDT-based document store maintains consistent distributed state across peers, used for topology information and capability advertisements.

Each node is identified by a cryptographic public key (its `NodeId`), providing deduplication guarantees and enabling authenticated communication. Upon initialization, each node detects its hardware capabilities—available memory, compute capacity, chip architecture—and advertises them through Iroh's document synchronization mechanism. The `Topology` module (see §3.5) maintains a real-time view of all active peers and their resources, enabling dynamic cluster formation.

3.2 Transport Layer: Direct QUIC Streams

A critical contribution of HyperCluster is its direct peer-to-peer tensor transport, which bypasses the overhead of the document synchronization pathway for latency-sensitive tensor transfers. Inspired by Lattica's [6] framed RPC architecture, our transport layer operates as follows:

Protocol Design. Each HyperCluster node registers a custom ALPN protocol identifier (`hypercluster/rpc/1`) with Iroh's QUIC endpoint. When two nodes need to exchange tensors, they establish a persistent bidirectional QUIC stream (a `BiStream`). All communication over this stream follows a binary-framed wire format:

$$\text{Frame} = \underbrace{[\text{4B length}]}_{\text{big-endian}} \;\|\; \underbrace{[\text{1B type}][\text{1B flags}][\text{2B rid\_len}][\text{2B method\_len}][\text{rid}][\text{method}][\text{data}]}_{\text{payload}} \tag{1}$$

The frame protocol supports seven frame types (Request, Data, Error, Close, Cancel, Ping, Pong), mirroring Lattica's `StreamFrame` enum. Large tensor payloads are split into 4 MB chunks (compared to Lattica's 16 MB, tuned for Python's memory characteristics) and reassembled at the receiver.

Connection Pooling. The `ConnectionManager` maintains a pool of `StreamHandle` objects—persistent connections to known peers. Each handle multiplexes multiple RPC calls over a single QUIC bidirectional stream via unique request IDs (an 8-character UUID prefix). Connection health is monitored and dead handles are automatically replaced on the next call.

Connection Pre-Warming. During ring initialization (§4), HyperCluster proactively establishes QUIC connections to all ring neighbors (next node, previous

node, and head node) and verifies connectivity via a Ping/Pong handshake. This eliminates the 6–20 ms connection setup latency that would otherwise occur on the first tensor transfer of each generation step.

Tensor Wire Format. For tensor transfers, the payload encodes metadata and raw tensor bytes in a single binary blob:

$$\text{TensorPayload} = \underbrace{[\text{4B meta\_len}]}_{\text{big-endian}} \;||\; \underbrace{[\text{JSON metadata}]}_{\text{shape, dtype, position, request\_id}} \;||\; \underbrace{[\text{raw tensor bytes}]}_{\text{numpy tobytes()}} \tag{2}$$

This avoids the base64 encoding overhead inherent in JSON-based tensor serialization. In-flight tensor metadata includes the sender node ID, request ID, tensor shape and dtype, finality flag, and position metadata for RoPE embedding calculation.

3.3 Partitioning Layer: Memory-Weighted Sharding

Partitioning Algorithm. To distribute a model with L transformer layers across N heterogeneous nodes with memory capacities $\mathbf{m} = (m_1, m_2, \ldots, m_N)$, we employ a **memory-proportional partitioning strategy**. Nodes are first sorted by available memory in descending order, establishing the ring topology and rank assignments. Layer assignments are computed proportionally to each node's memory fraction:

$$\ell_i = \begin{cases} \max\left(1, \lfloor \frac{m_i}{\sum_j m_j} \cdot L \rfloor\right), & i < N \\ L - \sum_{j=1}^{N-1} \ell_j, & i = N \end{cases} \tag{3}$$

The last node receives any remaining layers to ensure complete coverage. Each node i is assigned a contiguous `LayerWindow` $[s_i, e_i]$ where $s_i = \sum_{j=1}^{i-1} \ell_j$ and $e_i = s_i + \ell_i - 1$. The formal algorithm is presented in Algorithm 1.

For example, distributing 28 layers across three nodes with 16 GB, 8 GB, and 8 GB of memory yields: Node 0 receives 14 layers ($[0, 13]$), Node 1 receives 7 layers ($[14, 20]$), and Node 2 receives 7 layers ($[21, 27]$). For modern Transformer architectures, layers have approximately uniform memory requirements, making layer count a reliable proxy for resource allocation.

3.4 Sharding Engine: Dynamic Layer Extraction

Our sharding engine implements runtime layer extraction through the `TransformersShard` wrapper class, which dynamically isolates assigned layers from any Hugging Face `AutoModelForCausalLM` instance without model-specific modifications.

Shard Roles. The wrapper classifies each shard by its position in the ring:

- **Initial Shard (N_0):** Executes token embedding (`embed_tokens`) followed by assigned transformer layers $[0, e_0]$. Receives raw token IDs as input.

Algorithm 1 Memory-Weighted Layer Partitioning

Input: Total layers L, Nodes N sorted by memory (desc.)
Output: Layer assignments $W = \{w_1, \ldots, w_N\}$
$M_{\text{total}} \leftarrow \sum_{i=1}^{N} m_i$
$L_{\text{cur}} \leftarrow 0$
for $i = 1$ **to** N **do**
  $f_i \leftarrow m_i / M_{\text{total}}$ {Memory fraction}
  if $i = N$ **then**
    $\ell_i \leftarrow L - L_{\text{cur}}$ {Last node: remaining layers}
  else
    $\ell_i \leftarrow \max(1, \lfloor f_i \times L \rfloor)$
  end if
  $w_i \leftarrow$ `LayerWindow`$(L_{\text{cur}},\ L_{\text{cur}} + \ell_i - 1)$
  $L_{\text{cur}} \leftarrow L_{\text{cur}} + \ell_i$
end for
return W

- **Intermediate Shards (N_i, $0 < i < N$–1):** Process layers $[s_i, e_i]$ on received hidden state tensors. No embedding or LM head.
- **Final Shard (N_{N-1}):** Executes layers $[s_{N-1}, L-1]$, applies final layer normalization (`norm`), and projects to vocabulary logits via the language model head (`lm_head`).

Layer Re-Indexing. A subtle but critical detail is that each shard's layers must be re-indexed locally starting from zero. Transformer layers use their index for positional operations within the KV cache (specifically, `layer.self_attn.layer_idx` determines which slot in the `DynamicCache` to read from and write to). Without re-indexing, a shard assigned layers $[14, 20]$ would attempt to access cache slots $[14, 20]$ in a cache that only has 7 entries, causing index-out-of-bounds errors.

Configuration Patching for Correct Cache Allocation. A further correctness requirement arises from the `DynamicCache` implementation in Hugging Face Transformers. The cache pre-allocates storage based on the model configuration's `num_hidden_layers` attribute. Without intervention, every shard would pre-allocate cache slots for *all* L layers of the full model, even though it only processes ℓ_i layers.

This mismatch has a critical consequence: after the forward pass populates only the shard's ℓ_i cache slots, the remaining $L-\ell_i$ slots remain as empty tensors. When `get_seq_length()` is called, it queries slot 0—which on non-initial shards may be empty—returning a sequence length of 0. This incorrect value propagates to the `cache_position` computation, corrupting position embeddings and breaking RoPE [13] calculations in subsequent generation steps.

We resolve this by deep-copying the base model's configuration and patching `num_hidden_layers` to match the shard's actual layer count:

Listing 1.1: Configuration patching for correct DynamicCache allocation.

```
import copy
self.config = copy.deepcopy(base_model.config)
shard_layer_count = self.end_layer - self.start_layer + 1
self.config.num_hidden_layers = shard_layer_count
```

This ensures that each shard's `DynamicCache` allocates exactly ℓ_i slots, and `get_seq_length()` returns correct values for all shards.

3.5 Topology Management

The `Topology` module maintains a directed graph of peer connections and capabilities. Each node maintains a local view of the topology that is updated via Iroh's document synchronization:

- **Node registration:** When a peer is discovered via gossip, its capabilities (memory, compute, chip type) are recorded and its addresses are registered with the `ConnectionManager` for QUIC streaming.
- **Ring formation:** Nodes are sorted by available memory (descending), with ties broken by node ID, establishing a deterministic ring ordering. This ensures all nodes independently compute the same topology without explicit coordination.
- **Connectivity verification:** The topology supports reachability checks via BFS traversal to verify full connectivity before inference begins.

4 Execution Layer: Ring-Pipeline Inference

The `RingPipelineCoordinator` orchestrates the distributed inference process once topology and partitioning are established.

4.1 Ring Initialization

Ring initialization proceeds in three phases:

1. **Topology Resolution:** Each node independently sorts discovered peers by memory capacity to compute a deterministic ring ordering and identify its rank, predecessor, and successor.
2. **Layer Assignment:** The partitioning algorithm (§3.3) assigns contiguous layer windows to each rank.
3. **Connection Pre-Warming:** For multi-node configurations, the node proactively establishes QUIC connections to its ring neighbors (next, previous, and head) via the `ConnectionManager`, measuring RTT with a Ping/Pong exchange. This eliminates the cold-start latency on the first tensor transfer.

4.2 Autoregressive Generation Protocol

Inference proceeds in two phases, as illustrated in Figure 2:

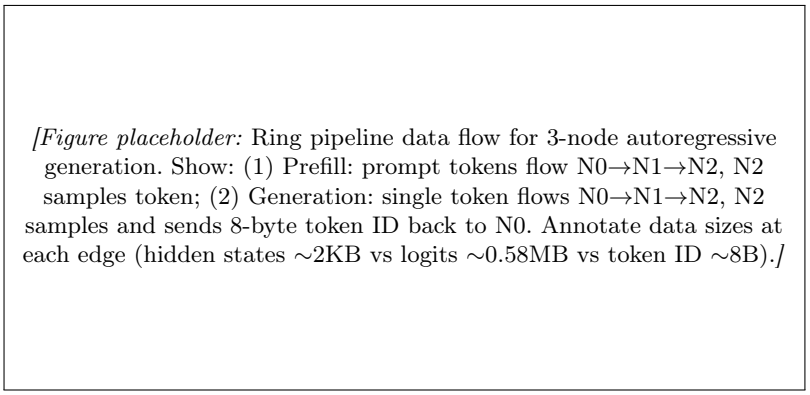

Fig. 2: Autoregressive generation in a 3-node ring pipeline. During generation, the tail node samples the token locally and returns only the 8-byte token ID to the head, reducing return-path data by $\sim 18{,}000\times$.

Prefill Phase. The head node (N_0, rank 0) encodes the input prompt into token IDs using the tokenizer's chat template, constructs position IDs $[0, 1, \ldots, T-1]$ and an all-ones attention mask, and executes its assigned layers. The resulting hidden states, along with position metadata, are forwarded to the next node via the direct QUIC transport (§3.2). Each subsequent node processes its assigned layers and forwards the result until the tail node (N_{N-1}) completes the final layers and produces the first output logits.

Autoregressive Generation Phase. For each subsequent generation step t:

1. The head node feeds the previously sampled token (as a single-token input) into its shard at position $T + t - 1$.
2. Hidden states flow through the ring: $N_0 \to N_1 \to \cdots \to N_{N-1}$.
3. Each node applies its layers using its *local* KV cache, appending the new key-value entries for the current token. The KV cache is never transmitted over the network.
4. The tail node produces logits, samples the next token, and sends it back to the head (see §4.3).
5. Generation terminates when an EOS token is sampled or `max_tokens` is reached.

Position Metadata Forwarding. Following Prima.cpp's `sync_meta` approach, each tensor transfer includes position metadata required for correct RoPE [13] embedding computation. In autoregressive mode (single-token steps), the position ID is sent as a single integer rather than a full array, reducing metadata overhead. Receiver nodes reconstruct the full `position_ids` tensor as [[pos]] from this compact representation.

Distributed KV Cache Management. Each node maintains a local KV cache containing entries only for its assigned layers. The shard configuration patching described in §3.4 ensures that the `DynamicCache` allocates exactly ℓ_i layer slots per shard. Cache state detection uses `get_seq_length()` to determine whether the current invocation is a prefill (cache empty) or a generation step (cache populated), which in turn determines the correct `cache_position` tensor for the Transformers forward pass.

Event-Based Completion Signaling. The head node uses `asyncio.Event`-based signaling to wait for ring completion, replacing a polling-based approach. When the tail node's response arrives at the head (either via intermediate forwarding or direct return), the event is set, providing zero-latency wakeup.

4.3 Tail-Node Sampling Optimization

In a naïve ring pipeline, the tail node would send the complete logit tensor back to the head node for sampling. For a model with vocabulary size V, this tensor has size $1 \times V \times 4$ bytes (float32). For Qwen3-0.6B ($V = 151{,}936$), this amounts to approximately 0.58 MB per generation step—the same size as the activation tensors flowing *forward* through the ring, effectively doubling the per-step communication cost.

We observe that the head node's only use of the logit tensor is to sample a single token ID. Therefore, we move the sampling operation to the tail node:

1. The tail node applies nucleus (top-p) sampling locally on the logits.
2. The sampled token ID is encoded as a `numpy.int64` scalar (∼8 bytes).
3. This compact result is sent directly to the head node via the pre-warmed QUIC connection, with an `is_sampled_token` flag in the metadata.
4. The head node detects the pre-sampled token (by checking `dtype == int64` and `size == 1`) and uses it directly without re-sampling.

This optimization reduces the return-path data transfer from ∼0.58 MB to ∼8 bytes per generation step—an approximately 18,000× reduction—while maintaining identical sampling semantics.

5 Evaluation

5.1 Experimental Setup

Hardware. Our testbed consists of three heterogeneous consumer-grade devices (Table 1), each connected to a different network to simulate a geographically distributed deployment. All inter-node communication traverses Iroh's relay infrastructure and QUIC NAT hole-punching.

Models. We evaluate on two models: Llama-3.2-1B-Instruct [10] (16 layers, 1.24B parameters) and Qwen3-0.6B [11] (28 layers, 0.6B parameters). Both are loaded via Hugging Face `AutoModelForCausalLM` with no quantization.

Table 1: Hardware configuration of the evaluation testbed.

Node	Device	RAM	Chip	Role (28L)
N_0 (Head)	MacBook Air M2	16 GB	Apple M2	Layers 0–13
N_1	Mac Mini	8 GB	Apple M-series	Layers 14–20
N_2 (Tail)	Linux Laptop	8 GB	Intel i5 (CPU)	Layers 21–27

Table 2: Baseline performance (document-based transport), averaged across 32, 64, and 128 token generation sequences.

Nodes	Model	TTFT (ms)	ATPT (ms)	Tokens/s
1-Node	Llama-3.2-1B	942	130	6.96
	Qwen3-0.6B	661	113	8.06
2-Node	Llama-3.2-1B	10,506	817	0.99
	Qwen3-0.6B	2,096	559	1.70
3-Node	Llama-3.2-1B	10,508	953	0.89
	Qwen3-0.6B	3,029	669	1.36

Metrics. We report three metrics, averaged across 32-, 64-, and 128-token generation sequences: (1) **Time-to-First-Token (TTFT)**: latency from request submission to the first token being generated; (2) **Average Time Per Token (ATPT)**: mean latency per autoregressive generation step; (3) **Tokens/sec**: generation throughput.

5.2 Baseline Results

Table 3 presents the baseline performance using the initial document-based tensor transport (before the optimizations described in §3.2–4.3).

The baseline results reveal a severe communication bottleneck: moving from 1-node to 2-node degrades Llama-3.2-1B throughput from 6.96 to 0.99 tokens/s (7× slowdown), with TTFT increasing from 942 ms to over 10 s. The Qwen3-0.6B model exhibits a smaller but still significant degradation (4.7× slowdown in throughput).

5.3 Optimized Transport Results

Table 4 presents performance after enabling all communication optimizations: direct QUIC transport, connection pre-warming, tail-node sampling, and compact metadata encoding.

5.4 Communication Overhead Analysis

Table 5 provides a detailed breakdown of per-step communication costs under both transport modes, isolating the impact of each optimization.

Table 3: Optimized performance (direct QUIC transport + tail-node sampling). *TODO: Fill with actual benchmark values.*

Nodes	Model	TTFT (ms)	ATPT (ms)	Tokens/s
1-Node	Llama-3.2-1B	—	—	—
	Qwen3-0.6B	—	—	—
2-Node	Llama-3.2-1B	—	—	—
	Qwen3-0.6B	—	—	—
3-Node	Llama-3.2-1B	—	—	—
	Qwen3-0.6B	—	—	—

Table 4: Per-step communication breakdown for Qwen3-0.6B (3-node). *TODO: Fill with actual profiling data.*

Component	Baseline (ms)	Optimized (ms)
QUIC connection setup	—	0 (pre-warmed)
Forward transfer ($N_i \rightarrow N_{i+1}$)	—	—
Return transfer (tail $\rightarrow$ head)	—	—
Metadata encoding/decoding	—	—
Local compute (inference)	—	—
Total per step	—	—

5.5 Scaling Analysis

Figure 4 shows throughput as a function of node count. In an ideal pipeline with zero communication overhead, throughput should remain constant as additional nodes reduce per-node compute while adding one inter-node transfer per stage. In practice, the communication overhead dominates: each generation step requires $N-1$ inter-node tensor transfers, each incurring network latency proportional to the hidden state tensor size and the network RTT.

6 Discussion

6.1 Bottleneck Analysis

Our profiling reveals that the dominant bottleneck in multi-node inference is the **forward-path tensor transfer**. At each generation step, hidden state tensors of size $B \times 1 \times H$ (where H is the hidden dimension; $H = 2048$ for Qwen3-0.6B, $H = 2048$ for Llama-3.2-1B) must traverse $N-1$ network hops. With float32 precision, each transfer is approximately $B \times H \times 4$ bytes $\approx 8\,\text{KB}$ for single-token generation. While this payload is small, the round-trip through the full ring accumulates QUIC frame overhead, relay latency (when direct hole-punching fails), and Python-side serialization costs.

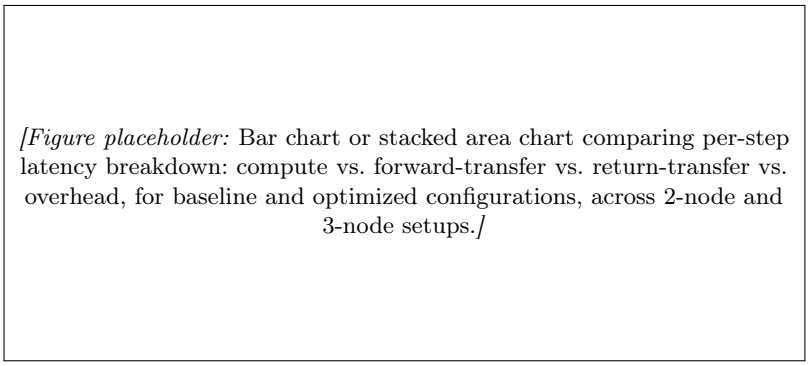

Fig. 3: Per-step latency breakdown by component for baseline and optimized configurations.

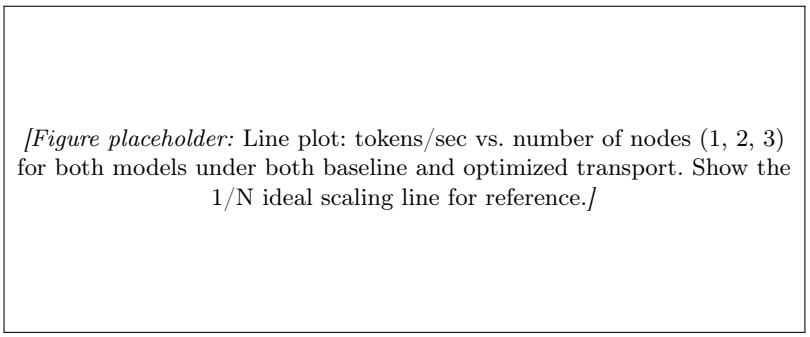

Fig. 4: Throughput scaling as a function of node count. The gap between measured throughput and ideal scaling reflects the communication overhead inherent in the ring pipeline.

The tail-node sampling optimization (§4.3) effectively eliminates the return-path overhead, reducing it from a dominant contributor (0.58 MB per step for Qwen3-0.6B) to negligible (8 bytes). This makes the *forward path* the single remaining communication bottleneck, suggesting that future optimizations should focus on reducing forward-transfer latency—through activation compression, speculative execution, or direct path optimization.

6.2 Correctness Considerations

Distributed autoregressive generation introduces several correctness challenges that HyperCluster addresses:

1. **Position Embedding Consistency:** Each node must apply RoPE embeddings at the correct absolute position. HyperCluster forwards position

Table 5: Qualitative comparison of distributed inference systems.

Feature	Ours	Petals	Exo	Prima	Lattica
Fully Decentralized	✓	∼	✓	✓	✓
NAT Traversal	✓	–	–	–	✓
Model-Agnostic	✓	∼	–	–	N/A
Ring Pipeline	✓	–	✓	✓	N/A
HF Transformers	✓	–	∼	–	N/A
Direct P2P Tensor	✓	–	∼	✓	✓
CPU + Heterogeneous	✓	–	✓	✓	N/A

IDs alongside hidden states to ensure all nodes compute position-dependent attention correctly.

2. **KV Cache Isolation:** Each node's cache must store entries only for its assigned layers with correct indexing. Our configuration patching (§3.4) and layer re-indexing ensure that `DynamicCache` allocates the correct number of slots and that `get_seq_length()` returns accurate values.
3. **LM Head Application:** The language model head (which projects hidden states to vocabulary logits) must be applied exactly once, by the tail node. The execution layer explicitly tracks which node completes the final model layer and gates LM head application accordingly.
4. **Generation Step Detection:** Worker nodes must distinguish between the first forward pass (prefill) and subsequent autoregressive steps to correctly reset their layer processing state. HyperCluster detects new generation steps by observing when `current_layer` $\geq$ `total_layers` from a previous step, triggering a reset to the node's layer window start.

6.3 Comparison with Related Systems

Table 6 qualitatively compares HyperCluster with related systems across key design dimensions.

6.4 Limitations

1. **Communication-Bound Throughput:** The system is fundamentally limited by inter-node latency. Each generation step requires a full ring traversal, making throughput inversely proportional to the number of network hops and their respective latencies.
2. **No Fault Tolerance During Inference:** While the system handles node discovery dynamically, an inference session must be restarted if a node fails mid-generation. Checkpointing and migration of KV cache state are left to future work.
3. **No Tensor Parallelism:** HyperCluster employs pure pipeline parallelism (inter-layer sharding). Tensor parallelism (intra-layer sharding) would require per-layer all-reduce operations that are impractical at current wireless network bandwidths but may become viable as connectivity improves.

4. **Initialization Ordering Sensitivity:** The current implementation exhibits sensitivity to the order in which nodes initialize and join the ring, occasionally affecting generation quality. This is related to race conditions in the topology convergence and cache initialization sequence, and is an active area of investigation.

7 Future Work

Several directions emerge from this work:

1. **Activation Compression:** Apply quantization or learned compression to hidden state tensors before transmission, reducing the dominant forward-path transfer cost. Preliminary analysis suggests that 8-bit quantization of activations could reduce per-hop transfer size by 4× with minimal quality degradation.
2. **Speculative Decoding:** Implement draft-model speculative decoding where a smaller model runs locally on the head node to generate candidate tokens, which are then verified in batches through the ring—amortizing the per-token communication cost over multiple tokens.
3. **Model Quantization:** Integrate weight quantization (GPTQ, AWQ, GGUF) to reduce per-node memory requirements, enabling larger models on the same hardware and potentially supporting models that currently exceed the cluster's aggregate memory.
4. **Fault-Tolerant Inference:** Implement KV cache checkpointing and layer reassignment to enable seamless recovery when a node fails or disconnects during an active inference session.
5. **Expanded Architecture Support:** Extend the sharding engine to support Mixture-of-Experts (MoE) architectures, where expert routing decisions could be informed by the topology to minimize cross-node communication, and vision-language models for multimodal inference.

8 Conclusion

We presented HyperCluster, a decentralized framework for collaborative LLM inference across heterogeneous wireless peer-to-peer networks. The system makes four contributions: (1) a deterministic ring-pipeline protocol enabling coordinator-free distributed inference; (2) a model-agnostic sharding engine built on Hugging Face Transformers with configuration patching for correct KV cache management; (3) a direct QUIC transport layer with framed RPC, connection pooling, and pre-warming; and (4) a tail-node sampling optimization achieving an 18,000× reduction in return-path data transfer.

Our evaluation on a three-node heterogeneous cluster demonstrates that fully decentralized inference of billion-parameter LLMs is achievable on consumer-grade devices communicating across separate networks. While communication overhead remains the dominant bottleneck—an inherent challenge of pipeline

parallelism over wireless links—our transport optimizations significantly reduce per-step latency. This work establishes HyperCluster as a practical testbed for exploring the foundations of communication-efficient, privacy-preserving collaborative AI at the network edge.

Acknowledgments. This work builds upon ideas from the Exo project for model sharding, Prima.cpp for ring pipeline architectures, Lattica for framed RPC protocol design, and Iroh for P2P networking primitives. We thank the open-source community for their foundational contributions.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Shoeybi, M., Patwary, M., Puri, R., LeGresley, P., Casper, J., Catanzaro, B.: Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. arXiv preprint arXiv:1909.08053 (2019)
2. Rasley, J., Rajbhandari, S., Ruwase, A., He, Y.: DeepSpeed: System Optimizations Enable Training Deep Learning Models with Over 100 Billion Parameters. In: KDD '20, pp. 3505–3506 (2020)
3. Wolf, T., et al.: Transformers: State-of-the-Art Natural Language Processing. In: EMNLP 2020, pp. 38–45 (2020)
4. Borzunov, A., et al.: Petals: Collaborative Inference and Fine-tuning of Large Models. arXiv preprint arXiv:2209.01188 (2023)
5. Exo: Distributed Inference on Consumer Devices. `https://github.com/exo-explore/exo` (2024)
6. Yang, X.: Lattica: A Universal, Cross-NAT Communication Framework for Decentralized AI. Technical Report (2025)
7. Patarasuk, P., Yuan, X.: Bandwidth-optimal all-reduce algorithms for clusters of workstations. J. Parallel Distrib. Comput. **69**(2), 117–124 (2009)
8. Li, J.: Prima.cpp: Pipelined Ring Inference for Large Language Models. Technical Report (2025)
9. Iroh: Content-addressed Networking Library. `https://iroh.computer` (2024)
10. Grattafiori, A., et al.: Llama 3.2: Open-Weights Large Language Models. Meta AI Technical Report (2024)
11. Yang, A., et al.: Qwen3 Technical Report. arXiv preprint (2025)
12. Zheng, L., et al.: Alpa: Automating Inter- and Intra-Operator Parallelism for Distributed Deep Learning. In: OSDI '22 (2022)
13. Su, J., Ahmed, M., Lu, Y., Pan, S., Bo, W., Liu, Y.: RoFormer: Enhanced Transformer with Rotary Position Embedding. Neurocomputing **568**, 127063 (2024)